

Reviewer Pack — Free Entropy Lower Bound for Disjointness Communication Comp...

Entry 5f1c06d77908 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-20 14:39:15 UTC

Free Entropy Lower Bound for Disjointness Communication Complexity

Entry ID: 5f1c06d77908 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-20 14:39:15 UTC

1. Conjecture

Field A (mathematical branch): Free Probability **Field B** (complexity object): Communication Complexity of Disjointness

Statement:

For a random 3-CNF instance M with n variables, the free entropy $\phi(M)$ of its adjacency matrix satisfies $\phi(M) \geq c \cdot n$ for some universal constant $c > 0$. The conjecture is falsified if any instance with $n \leq 40$ has $\phi(M) < c \cdot n$.

Rationale (proposer's reasoning):

Free entropy quantifies the 'non-commutative randomness' of matrices, capturing structural complexity that may correlate with communication requirements. Disjointness's $\Omega(n)$ lower bound suggests a potential link between matrix randomness and protocol cost.

Taxonomy category: COMM_DISJ (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): bd9fac9ca390a4bc

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
KARP_LIPTON	SAFE	0.95	SAFE	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_3cnf(n):
        clauses = []
        for _ in range(2 * n):
            clause = [random.randint(1, n), -random.randint(1, n)]
            if random.choice([True, False]):
                clause[0] *= -1
            if random.choice([True, False]):
                clause[1] *= -1
            clauses.append(clause)
        return clauses

    def adjacency_matrix(clauses, n):
        M = [[0] * n for _ in range(n)]
        for i, (x, y) in enumerate(clauses):
            x -= 1
            y -= 1
            M[x][y] = 1
            M[y][x] = 1
        return M

    def monte_carlo_free_entropy(M, n):
        def characteristic_polynomial(M):
            if len(M) == 0:
                return [1]
            if len(M) == 1:
                return [M[0][0], -1]
            det = 0
            for j in range(len(M)):
                M2 = [row[:j] + row[j+1:] for row in M[1:]]
                det += (-1)**j * M[0][j] * determinant(M2)
            return det

        def determinant(M):
            if len(M) == 0:
```

```

        return 1
    if len(M) == 1:
        return M[0][0]
    det = 0
    for j in range(len(M)):
        M2 = [row[:j] + row[j+1:] for row in M[1:]]
        det += (-1)**j * M[0][j] * determinant(M2)
    return det

def roots(poly):
    if len(poly) == 1:
        return []
    if len(poly) == 2:
        return [(-poly[1]) / poly[0]]
    a, b, c = poly
    discriminant = b**2 - 4*a*c
    if discriminant < 0:
        return []
    sqrt_discriminant = math.sqrt(discriminant)
    roots = [(-b + sqrt_discriminant) / (2*a), (-b - sqrt_discriminant) / (2*a)]
    return roots

poly = characteristic_polynomial(M)
roots_poly = roots(poly)
free_entropy = sum(math.log(abs(z)) for z in roots_poly if abs(z) > 0) / n
return free_entropy

n = 40
M = adjacency_matrix(generate_3cnf(n), n)
phi_M = monte_carlo_free_entropy(M, n)

return {
    "metric_name": "free_entropy",
    "metric_value": phi_M,
    "instances_tested": 1,
    "conjecture_holds": phi_M >= 0.2 * n,
    "counterexample": "" if phi_M >= 0.2 * n else f"Graph with n={n}, phi(M)={phi_M}"
}

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] or [random.randint(2, 97) for _ in range(30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):

```

```

    first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"phi(M) < 0.2n\" first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE no_counterexamples_found")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_bbc2ea2d.py", line 100, in <module>
    result = run_trial(seed)
              ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_bbc2ea2d.py", line 83, in run_trial
    M = adjacency_matrix(generate_3cnf(n), n)
        ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_bbc2ea2d.py", line 37, in adjacency_matrix
    M[x][y] = 1
    ~~~
IndexError: list index out of range

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed with IndexError before producing data, preventing evaluation of the conjecture. | next: Fix the adjacency_matrix implementation to handle index ranges properly and re-run tests

11. Audit log (LLM calls)

Total LLM calls: 14

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	136709
2	propose	ollama_remote	qwen3:8b	0	0	102556
3	propose	ollama_remote	qwen3:8b	0	0	138264
4	propose	ollama_local	qwen3:8b	0	0	229214
5	propose	ollama_remote	qwen3:8b	0	0	138496
6	propose	ollama_remote	qwen3:8b	0	0	74719
7	preregistration	ollama_remote	qwen3:8b	0	0	31687
8	novelty	ollama_remote	qwen3:8b	0	0	27316
9	novelty	ollama_remote	qwen3:8b	0	0	20352

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14326
11	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8322
12	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8901
13	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11225
14	judge	ollama_remote	qwen3:8b	0	0	21117

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 963204 ms total latency. Provider mix: {'ollama_remote': 13, 'ollama_local': 1}

(full prompt+response transcripts available in *research/audit/5f1c06d77908.jsonl*)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in *research/audit/5f1c06d77908.jsonl* for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: *research/pvsnp_sandbox/test_*.py* (per-cycle)

Replay tarball: *research/replay/5f1c06d77908.tar.gz* (if generated)